Paper Presentation on

# Open-World Task and Motion Planning via Vision-Language Model Inferred Constraints

Boxiang (William) Fu

16-832 (Spring 2026) Integrated Planning and Learning
Carnegie Mellon University

**01/28/2026**

# Paper to Cover

- **Title**: Open-World Task and Motion Planning via Vision-Language Model Inferred Constraints
- **Authors**: Nishanth Kumar, William Shen, Fabio Ramos, Dieter Fox, Tomás Lozano-Pérez, Leslie Pack Kaelbling, and Caelan Reed Garrett
- **Affiliation**: MIT CSAIL and NVIDIA Research
- **Year**: 2025
- **Website**: https://nishanthjkumar.com/owl-tamp/
- **ArXiv**: https://arxiv.org/abs/2411.08253

# TLDR Summary

- Describes a way to combine Vision-Language Models (VLMs) with classical Task and Motion Planning (TAMP) by having the VLM generate task-specific constraints from natural language prompt, which a TAMP system then plans and executes.
- Solves the problem of VLMs performing poorly in long-horizon manipulation tasks.

# Motivation

- Two mainstream methods for manipulation:
- **Classical**: Task and Motion Planning (TAMP)
  - Method: Symbolic goal expression of planned tasks
  - Advantages: Capable of solving specific long horizon complex tasks
  - Problem: Cannot perform tasks for which their model is unspecified (e.g. open world scenarios)
- **New**: Vision-Language Models (VLMs)
  - Method: LLM or Neural-Network architecture using datasets
  - Advantages: Can adapt and solve a wide variety of short horizon tasks
  - Problem: Performs poorly over long horizon periods

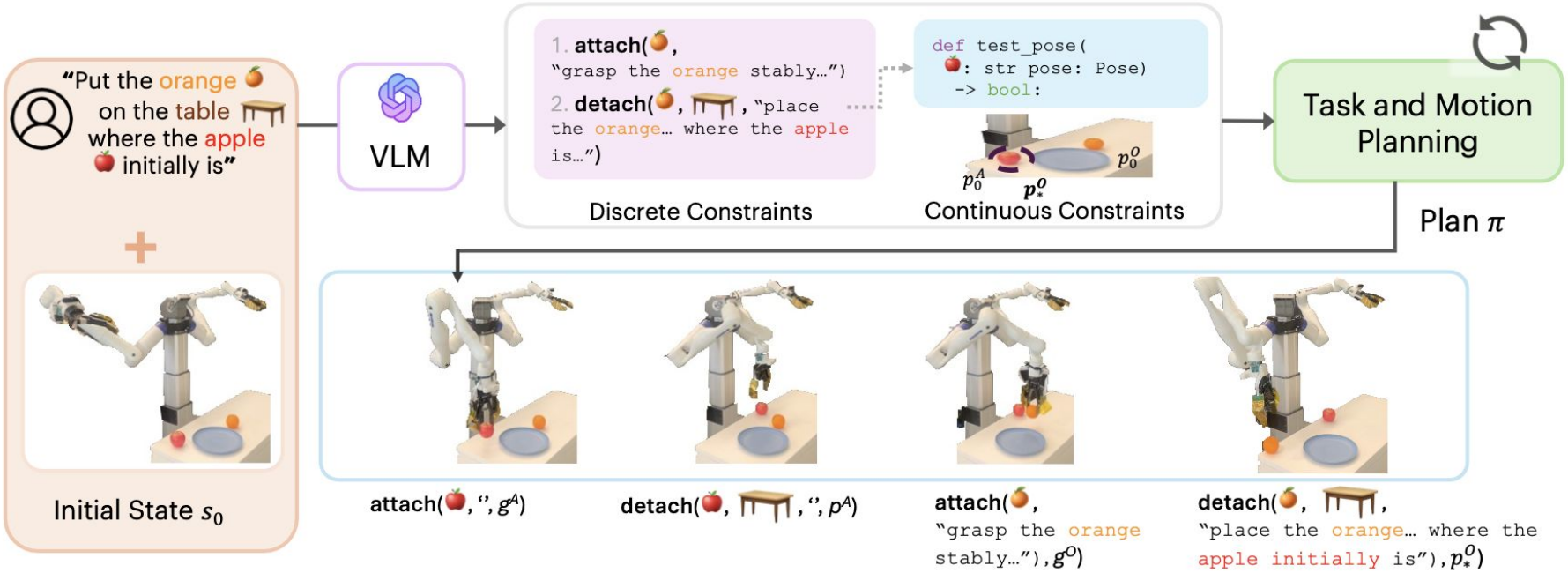
# Related Work

- **Task and Motion Planning (TAMP)**: Symbolic goal expression of planned tasks. Classical approach for long-horizon robot manipulation tasks
- **LLMs/VLMs (Task Planning)**: Generate symbolic plans from natural language. Utilize low level executor to execute the generated symbolic plan
- **Code as Policy (CoP)**: Bypasses explicit symbolic planning. Generates executable programs that directly compose low-level skills into a policy
- **LLMs/VLMs (End-to-End)**: Directly generate motor commands from input natural language prompt. Currently very bad for long-horizon tasks

# OWL-TAMP

- Fuse together the “best of both worlds”
- Integrate TAMP and VLMs through the use of “constraints”
- VLM infers from natural language and tells TAMP what plans or parameter values are allowed
- Provides logical structure of the long horizon plan
- It does not directly control the robot how to move – this is still controlled by the TAMP controller
- For example:
  - “Cook the strawberry by putting it in the pan, then serve it in the bowl”
  - detach(strawberry, pan) **must occur before** detach(strawberry, bowl)

# OWL-TAMP


"Put the orange on the table where the apple initially is"
Initial State $s_0$
VLM
1. attach( , "grasp the orange stably…")
2. detach( , , "place the orange… where the apple is…")
Discrete Constraints
def test_pose( : str pose: Pose) -> bool:
$p_0^A$
$p_*^O$
$p_0^O$
Continuous Constraints
Task and Motion Planning
Plan $\pi$
attach( , '', $g^A$)
detach( , , '', $p^A$)
attach( , "grasp the orange stably…"), $g^O$)
detach( , , "place the orange… where the apple initially is"), $p_*^O$)

# Assumptions

1. Tasks are specified by the user in unambiguous natural language
2. Scene is static with no movement other than the robot
3. An off-the-shelf VLM/LLM model that can understand sequential partial tasks and convert it to pseudocode with relatively good accuracy
4. An off-the-shelf TAMP model which constraints commonplace manipulation primitives applicable across a wide range of tasks

# Methodology

1. Method for generating constraints on action sequences to specify partial plans with language descriptions
2. Method for generating constraints on continuous variables affected within the partial plan from (1)
3. Combining (1) and (2) within a TAMP system

# Methodology – Step 1

- Prompt VLM to generate a partial plan of discrete constraints for the TAMP
- Problem: VLMs hallucinate and can provide impossible solutions
- Solution:
  - Ground the set of actions and literals (True/False states of the world) to only ones that are reachable
  - Prompt the VLM to only return solutions in this grounded set

**Algorithm 1** VLM Task Reasoning

**procedure** VLM-TASK-REASONING($s_0, \mathcal{A}, g$)
  $A \leftarrow$ GROUND-ACTIONS($s_0, \mathcal{A}$)
  $L \leftarrow s_0 \cup \{l \mid a \in A.\ l \in e.\mathit{eff}\}$
  $[a_1, ..., a_n, l_1, .., l_m] \leftarrow$ QUERY-VLM("What partial plan using actions $\{A\}$ for goal literals $\{L\}$ achieves goal $\{g\}$?")
  **for** $i \in [1, n-1]$ **do**
    $a_i.\mathit{eff} \leftarrow a_i.\mathit{eff} \cup \{\texttt{Executed}(i)\}$
    $a_{i+1}.\mathit{pre} \leftarrow a_{i+1}.\mathit{pre} \cup \{\texttt{Executed}(i)\}$
  $a_n.\mathit{eff} \leftarrow a_n.\mathit{eff} \cup \{\texttt{Executed}(n)\}$
  $G \leftarrow \{l_1, .., l_m\}$
  **return** SOLVE-TAMP($s_0, A, G \cup \{\texttt{Executed}(n)\}$)

**Legend:**

- *s*: state
- *A*: action set
- L: literal set
- *g*: goal
- *.eff*: effect
- *.pre*: predicate
- *G*: partial plan

# Methodology – Step 2

- Turn the partial plan generated from Step 1 into a full legal plan that TAMP can use
- Problem: Partial plan from Step 1 could be open-ended natural language

```
detach("place the mug stably on the table
        ensuring it is upright and positioned
        to receive the coffee", mug, ...),
```

- Solution:
  - Introduce a predicate (boolean) function that converts the natural language into code that can be tested

# Methodology – Step 2

```
def test_poses(p) -> bool:
    ontop_table_bounds =
        modify_pose_bounds_to_be_ontop
        _of_object('mug', 'table')
    mug_on_table =
        position_within_bounds(mug.pose,
        ontop_table_bounds)
    upright_orientation = abs(mug.pose.roll)
        < 0.1 and abs(mug.pose.pitch) < 0.1
    return mug_on_table and
        upright_orientation
```

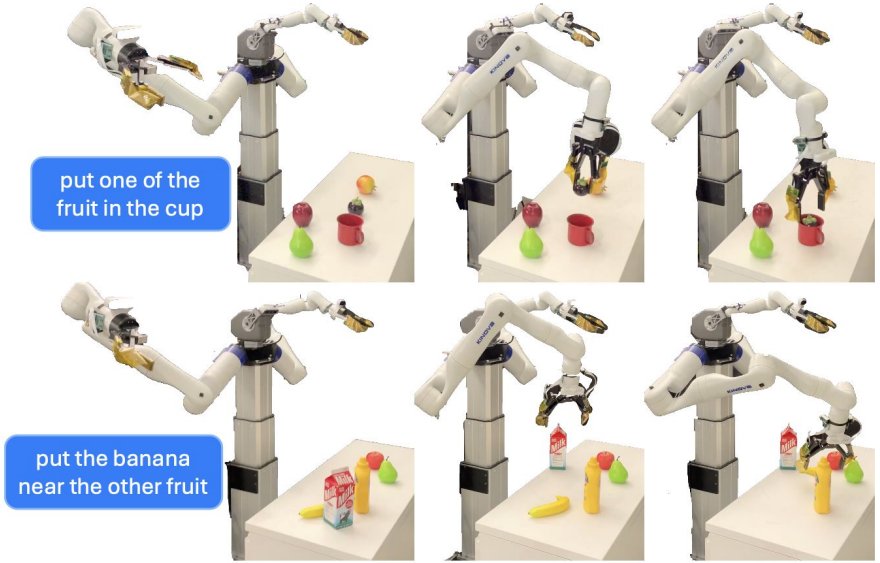
- Once the full plan is sent to the TAMP planner, it can check against this test to ensure the generated plan satisfies the original natural language partial plan
- Forces TAMP system to output solutions given this constraint

# Methodology – Step 3

- Standard search-then-sample TAMP
- **Search Phase**: A* search over action sequences to generate symbolic task plan
- **Sample Phase**: Hand-crafted sampling of continuous environment and manipulator configurations to check if pose is valid or move is legal
- **Backtracking**: If sampling fails, backtrack to search phase to try a different skeleton plan
- **Result**: Terminates when the sampling phase passes all checks

# Experiments

- **Tasks**: 10 of varying difficulty, all require multi-sequence planning
  - Example: "*I want to pour some coffee into the cup; can you set up the cup on the table so I can do this properly?*"
- **Environment**: Two Kinova Gen3 manipulator arms
- **Items**: YCB dataset
- **OD**: Grounding DINO
- **VLM**: GPT-4o (CoT)

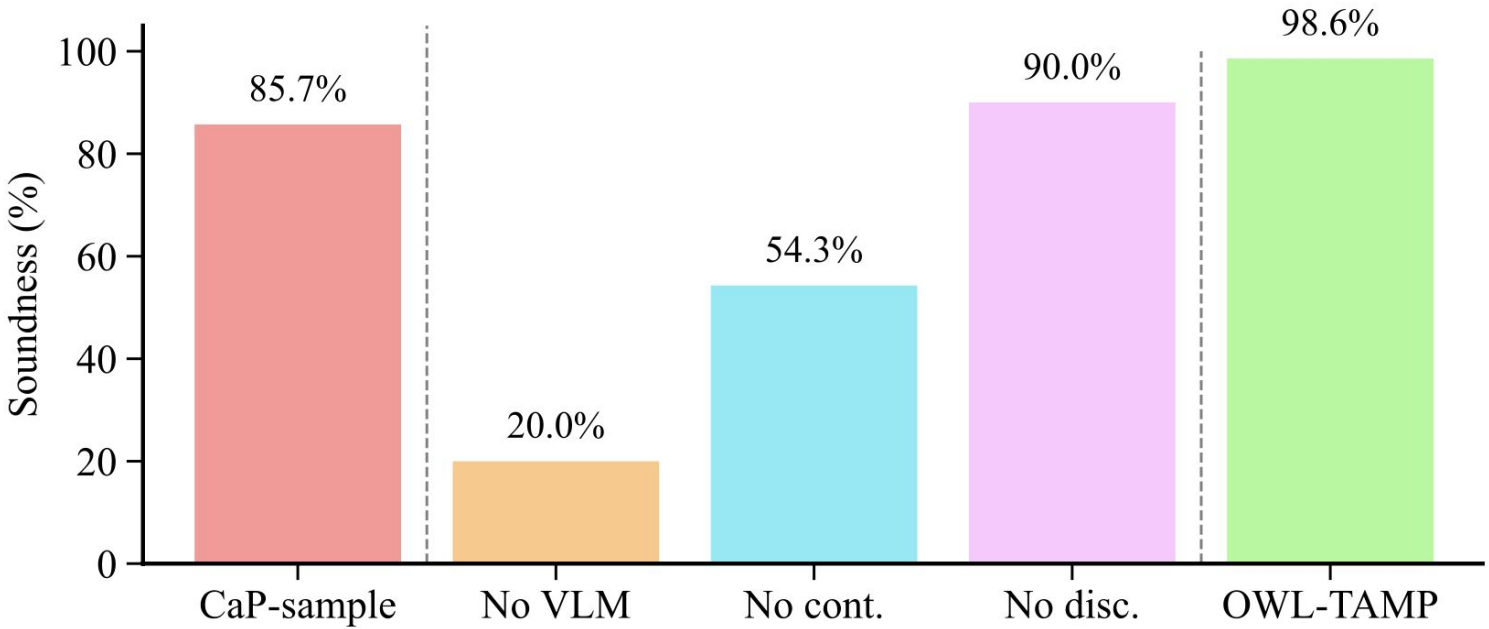


# Results – Success Rate

| | Tasks | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|
| **Method** | Berry1 | Citrus | Berry2 | BerryCook | FruitSort | Coffee | Mug1 | Mug2 | Mug3 | SoupPour | **Overall** |
| CaP | 0% | 0% | 0% | 0% | 0% | 0% | 0% | 0% | 0% | 0% | 0% |
| CaP-sample | **100%** | 20% | 20% | 0% | 0% | 0% | 0% | 0% | 0% | 0% | 14% |
| No VLM | **100%** | **100%** | **100%** | 0% | 0% | 0% | 0% | 0% | 20% | 0% | 32% |
| No sample | 0% | 0% | 0% | 0% | 0% | 0% | 0% | 0% | 0% | 0% | 0% |
| No disc. | **100%** | **100%** | **100%** | 0% | 0% | 0% | 0% | 0% | 20% | 0% | 32% |
| No cont. | **100%** | **100%** | **100%** | **100%** | 10% | 60% | 70% | 0% | 0% | 20% | 56% |
| No back. | **100%** | **90%** | **100%** | 60% | 80% | **100%** | 40% | 30% | 0% | 0% | 60% |
| OWL-TAMP | **100%** | **100%** | **100%** | 60% | **100%** | **100%** | **100%** | **100%** | **70%** | **90%** | **92%** |

- **CaP**: Code as Policy

# Results – Soundness




- **Soundness**: Higher rates indicate lower false positives (i.e. algorithm saying natural language goal was completed but in reality it did not)

# Results – Wall Clock Time

| Method | Tasks | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| | Berry1 | Citrus | Berry2 | BerryCook | FruitSort | Coffee | Mug1 | Mug2 | Mug3 | SoupPour |
| CaP | 10.03 ±0.32 (47.38%) | 21.27 ±1.10 (25.02%) | 12.63 ±1.13 (42.76%) | 13.77 ±0.95 (45.88%) | 19.50 ±0.67 (26.14%) | 9.66 ±0.80 (47.61%) | 19.77 ±2.84 (46.16%) | 23.25 ±1.65 (30.42%) | 16.37 ±1.49 (38.14%) | 19.33 ±1.17 (42.88%) |
| CaP-sample | 22.40 ±12.70 (21.22%) | 248.49 ±54.38 (2.14%) | 245.31 ±35.87 (2.20%) | 262.43 ±14.12 (2.41%) | 20.88 ±2.05 (24.41%) | 40.39 ±16.21 (11.39%) | 281.74 ±2.99 (3.24%) | 75.61 ±78.73 (9.35%) | 274.50 ±11.97 (2.27%) | 187.90 ±83.47 (4.41%) |
| No VLM | 11.90 ±1.35 (8.85%) | 41.59 ±4.85 (4.53%) | 47.66 ±3.20 (1.60%) | 15.96 ±1.23 (29.77%) | 16.56 ±0.14 (24.60%) | 4.92 ±0.04 (10.25%) | 59.43 ±31.91 (4.11%) | 78.72 ±18.85 (6.36%) | 36.03 ±11.85 (8.04%) | 21.28 ±2.54 (20.83%) |
| No sample | 44.00 ±4.03 (85.97%) | 75.17 ±5.26 (72.29%) | 59.47 ±4.69 (83.77%) | 54.93 ±4.68 (82.13%) | 81.19 ±10.40 (74.17%) | 39.95 ±4.28 (85.49%) | 55.07 ±3.43 (73.59%) | 80.00 ±9.63 (69.38%) | 46.22 ±2.57 (70.58%) | 91.46 ±29.02 (82.12%) |
| No disc. | 44.92 ±4.07 (70.03%) | 91.62 ±6.22 (50.92%) | 80.87 ±6.83 (33.37%) | 44.50 ±4.38 (69.50%) | 36.10 ±2.01 (59.87%) | 18.92 ±2.96 (72.29%) | 118.42 ±16.71 (20.71%) | 120.94 ±9.99 (29.63%) | 81.85 ±10.58 (26.85%) | 96.95 ±6.37 (72.54%) |
| No cont. | 18.15 ±1.84 (40.92%) | 54.53 ±13.09 (16.21%) | 36.91 ±5.61 (29.48%) | 24.30 ±1.91 (29.40%) | 57.03 ±9.88 (15.68%) | 25.73 ±4.13 (29.93%) | 52.26 ±16.08 (14.61%) | 136.88 ±5.72 (5.72%) | 102.53 ±22.20 (6.04%) | 91.00 ±20.21 (9.42%) |
| No back. | 48.69 ±3.40 (71.92%) | 106.36 ±24.85 (60.35%) | 101.84 ±34.28 (72.59%) | 67.94 ±15.09 (75.11%) | 149.53 ±39.78 (66.52%) | 57.68 ±8.20 (65.30%) | 87.31 ±21.78 (58.23%) | 106.82 ±6.13 (54.49%) | 69.59 ±3.02 (52.24%) | 111.50 ±32.13 (67.36%) |
| OWL-TAMP | 46.92 ±3.37 (74.64%) | 108.50 ±24.61 (59.15%) | 101.88 ±34.47 (72.56%) | 71.38 ±15.31 (71.49%) | 154.32 ±43.43 (64.46%) | 57.96 ±7.69 (64.98%) | 113.18 ±28.50 (44.92%) | 140.51 ±23.06 (41.43%) | 118.38 ±23.74 (30.71%) | 173.81 ±49.00 (43.21%) |
| Manual | 13.53 ±2.36 (0.00%) | 44.42 ±5.34 (0.00%) | 24.84 ±2.78 (0.00%) | 19.78 ±1.34 (0.00%) | 49.72 ±3.59 (0.00%) | 22.15 ±6.36 (0.00%) | 45.18 ±16.74 (0.00%) | 79.91 ±33.04 (0.00%) | 37.64 ±10.40 (0.00%) | 81.38 ±41.85 (0.00%) |

# Limitations

- There is no provision for recovering from errors generated by the VLM (Step 1 and Step 2)
- There is no feedback loop between the VLM and TAMP. If the TAMP system declares a problem infeasible, a feedback loop would allow the VLM to replan a more feasible solution
- Hard to get a large dataset of robot manipulation tasks to finetune the VLM model
- If the algorithm returns successfully finished, it is not guaranteed that the actual result indeed satisfies the criteria specified in the original natural language goal
- No theoretical guarantees of solution quality
- Takes a very long time (buried deep in the appendix...)

# Conclusion

- Authors propose OWL-TAMP, integrating VLM with TAMP by using language to generate discrete and continuous planning constraints
- Merges the best of both worlds, essentially assigning the “reasoning” to the VLM, and long-horizon planning to the TAMP
- Results shows OWL-TAMP can solve a wider range of multi-step problems compared to the benchmark
- Some limitations of the algorithm include no recovery procedures, lack of quality data, no theoretical guarantees and long runtime complexity

# Thank You!